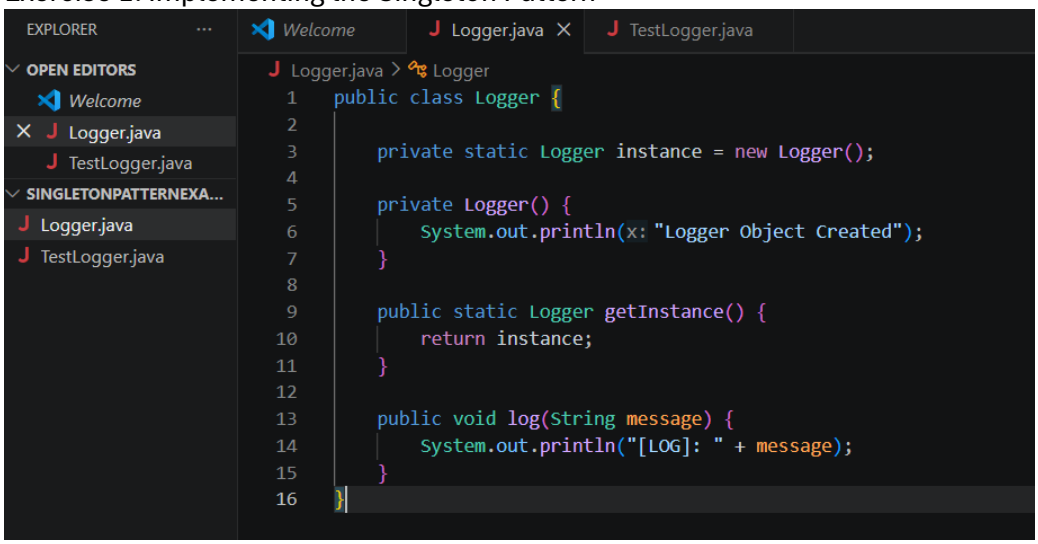
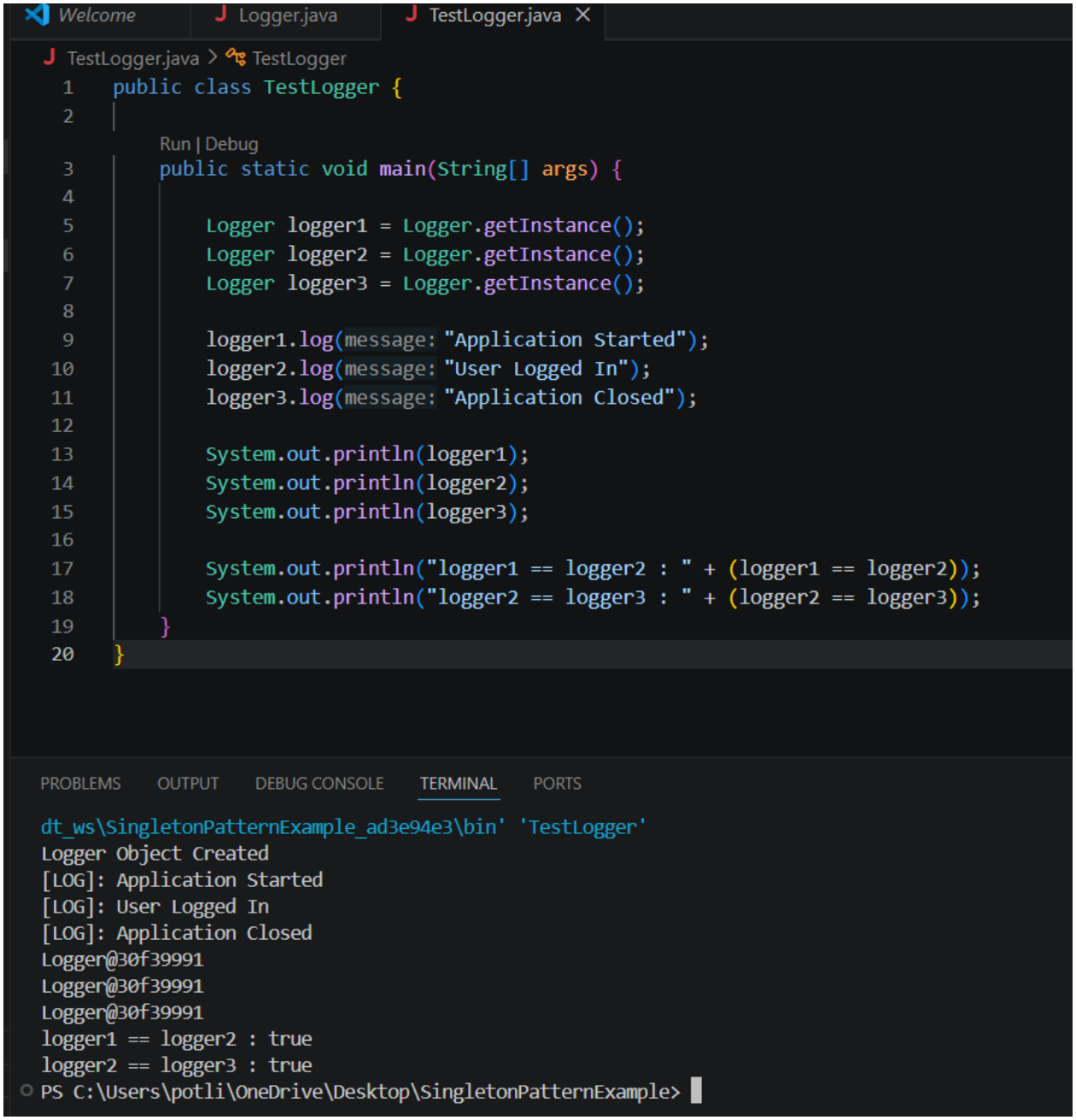
WEEK-1

Design Patterns & Principles

Exercise 1: Implementing the Singleton Pattern





Exercise 2: Implementing the Factory Method Pattern

Document.java > Document

```java
public interface Document {

    void open();

}
```

DocumentFactory.java > DocumentFactory

```java
public abstract class DocumentFactory {

    public abstract Document createDocument();

}
```

WordDocument.java > WordDocument

```java
public class WordDocument implements Document {

    @Override
    public void open() {
        System.out.println(x: "Opening Word Document");
    }
}
```

WordDocumentFactory.java > WordDocumentFactory

```java
public class WordDocumentFactory
        extends DocumentFactory {

    @Override
    public Document createDocument() {

        return new WordDocument();

    }
}
```

PdfDocument.java > PdfDocument

```java
public class PdfDocument implements Document {

    @Override
    public void open() {
        System.out.println(x: "Opening PDF Document");
    }
}
```

PdfDocumentFactory.java > PdfDocumentFactory

```java
public class PdfDocumentFactory
        extends DocumentFactory {

    @Override
    public Document createDocument() {

        return new PdfDocument();

    }
}
```

ExcelDocument.java > ExcelDocument

```java
public class ExcelDocument implements Document {

    @Override
    public void open() {
        System.out.println(x: "Opening Excel Document");
    }
}
```

ExcelDocumentFactory.java > ExcelDocumentFactory

```java
public class ExcelDocumentFactory
        extends DocumentFactory {

    @Override
    public Document createDocument() {

        return new ExcelDocument();

    }
}
```

FactoryTest.java > FactoryTest > main(String[])

```java
public class FactoryTest {
    Run | Debug
    public static void main(String[] args) {
        DocumentFactory wordFactory =
                new WordDocumentFactory();
        DocumentFactory pdfFactory =
                new PdfDocumentFactory();
        DocumentFactory excelFactory =
                new ExcelDocumentFactory();
        Document word =
                wordFactory.createDocument();
        Document pdf =
                pdfFactory.createDocument();
        Document excel =
                excelFactory.createDocument();
        word.open();
        pdf.open();
        excel.open();
    }
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\potli\
ws\FactoryMethodPattern_52051707\bin' 'FactoryTest'
PS C:\Users\potli\OneDrive\Desktop\FactoryMethodPattern>  c:; 
rograms\Eclipse Adoptium\jdk-17.0.10.7-hotspot\bin\java.exe' '
kspaceStorage\798b8e968c8fcad0e34ff6463db9f4a0\redhat.java\jdt_
Opening Word Document
Opening PDF Document
Opening Excel Document
```

Data structures and Algorithms

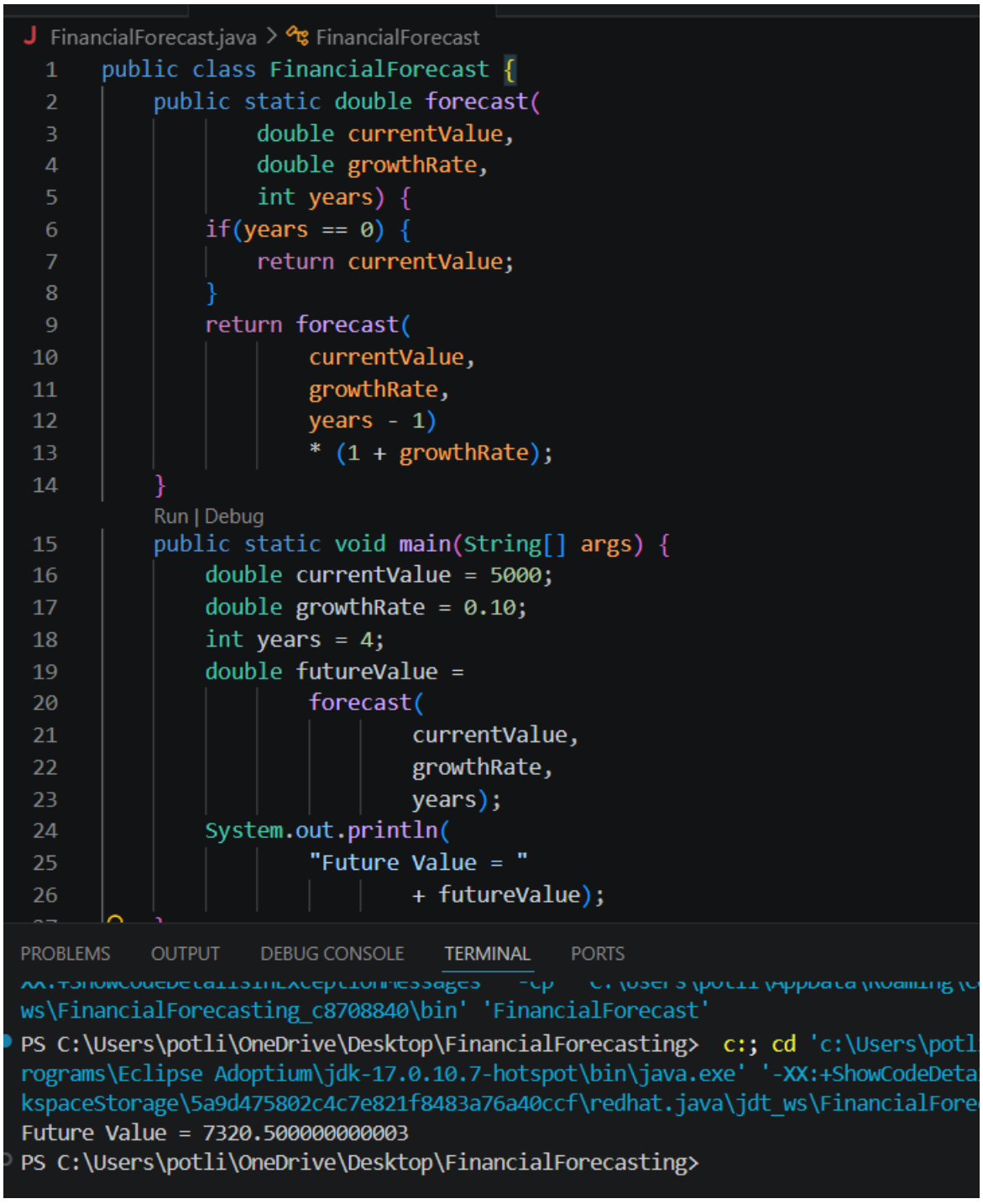
Exercise 2: E-commerce Platform Search Function

```
SearchTest.java > SearchTest
1   public class SearchTest {
        Run | Debug
2       public static void main(String[] args) {
3           Product[] products = {
4               new Product(productId: 101,
5                       productName: "Laptop",
6                       category: "Electronics"),
7               new Product(productId: 102,
8                       productName: "Phone",
9                       category: "Electronics"),
10              new Product(productId: 103,
11                      productName: "Shoes",
12                      category: "Fashion"),
13              new Product(productId: 104,
14                      productName: "Watch",
15                      category: "Accessories"),
16              new Product(productId: 105,
17                      productName: "Bag",
18                      category: "Fashion")
19          };
20          Product result1 =
21                  linearSearch(products, targetId: 104);
22          System.out.println(
23                  "Linear Search: " + result1);
24          Product result2 =
25                  binarySearch(products, targetId: 104);
26          System.out.println(
27                  "Binary Search: " + result2);
28      }
29      public static Product linearSearch(

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\potli\OneDrive\Desktop\ECommerceSearch>  c:; cd 'c:\Users
lipse Adoptium\jdk-17.0.10.7-hotspot\bin\java.exe' '-XX:+ShowCodeDeta
age\f0f7f860f0f1350b13e96e3c729565c8\redhat.java\jdt_ws\ECommerceSear
Linear Search: 104 Watch Accessories
Binary Search: 104 Watch Accessories
PS C:\Users\potli\OneDrive\Desktop\ECommerceSearch>
```

Exercise 7: Financial Forecasting



```java
public class FinancialForecast {
    public static double forecast(
            double currentValue,
            double growthRate,
            int years) {
        if(years == 0) {
            return currentValue;
        }
        return forecast(
                currentValue,
                growthRate,
                years - 1)
                * (1 + growthRate);
    }
    Run | Debug
    public static void main(String[] args) {
        double currentValue = 5000;
        double growthRate = 0.10;
        int years = 4;
        double futureValue =
                forecast(
                        currentValue,
                        growthRate,
                        years);
        System.out.println(
                "Future Value = "
                        + futureValue);
    }
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
ws\FinancialForecasting_c8708840\bin' 'FinancialForecast'
PS C:\Users\potli\OneDrive\Desktop\FinancialForecasting>  c:; cd 'c:\Users\potl
rograms\Eclipse Adoptium\jdk-17.0.10.7-hotspot\bin\java.exe' '-XX:+ShowCodeDeta
kspaceStorage\5a9d475802c4c7e821f8483a76a40ccf\redhat.java\jdt_ws\FinancialFore
Future Value = 7320.500000000003
PS C:\Users\potli\OneDrive\Desktop\FinancialForecasting>
```

PL/SQL

Exercise 1: Control Structures

[SQL Worksheet]*

```
1   SELECT * FROM Customers;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.012 seconds

	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1	1	Cheritha	65	15000	FALSE
2	2	Enosh	45	8000	FALSE
3	3	Rainy	70	20000	FALSE

	LOANID	CUSTOMERID	INTERESTRATE	DUEDATE
1	101	1	10	7/9/2026, 6:01:03 P
2	102	2	12	8/8/2026, 6:01:03 P
3	103	3	11	7/14/2026, 6:01:03

Query result | Script output | DBMS output | Explain Plan | SQL history

```
            SELECT CustomerID, Age
            FROM Customers;...
Show more...


Discount applied for Customer 1
Discount applied for Customer 3


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.015
```

Scenario 2:

Query result | Script output | DBMS output | Explain Plan | SQL history

```
            SELECT CustomerID, Balance
            FROM Customers;...
Show more...


Customer 1 promoted to VIP
Customer 3 promoted to VIP


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014
```

Scenario 3:

```
        CURSOR loan_cursor IS
            SELECT c.Name,...
Show more...


Reminder: Dear Cheritha, your Loan ID 101 is due on 09-JUL-2026
Reminder: Dear Rainy, your Loan ID 103 is due on 14-JUL-2026


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014
```

Exercise 3: Stored Procedures

	ACCOUNTID	ACCOUNTTYPE	BALANCE
1	101	Savings	10100
2	102	Savings	20200
3	103	Current	15000

Monthly Interest Applied Successfully.

SCENARIO 2:

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENT	SALARY
1	1	John	IT	55000
2	2	David	HR	40000
3	3	Smith	IT	66000

Bonus Updated Successfully.

SCENARIO 3:

	ACCOUNTID	ACCOUNTTYPE	BALANCE
1	101	Savings	9600
2	102	Savings	20700
3	103	Current	15000

Transfer Successful.